DepChain: A Dependable Permissioned Blockchain System

Rodrigo Perestrelo (ist1106074), Francisco Heleno (ist1106970), and Eduardo Silva (ist1106929)

Group 1
Instituto Superior Técnico, Universidade de Lisboa

Abstract. This report details the architecture and implementation of the DepChain project, a simplified permissioned blockchain system built on the Basic HotStuff consensus protocol. In its first stage, the system established a Byzantine Fault Tolerant (BFT) foundation using threshold signatures and authenticated perfect links. The first stage was subsequently strengthened with block-level signature verification, full blockchain recovery, and FIFO-ordered message delivery. In the second stage, we expanded the system into a fully functional distributed ledger by integrating the Hyperledger Besu Ethereum Virtual Machine (EVM). DepChain now supports native cryptocurrency (DepCoin) transfers, smart contract execution with a deterministic gas-fee mechanism, and transaction batching with fee-based ordering. Furthermore, we deploy a custom frontrunning-resistant ERC-20 token (ISTCoin). We present an explicit analysis of the threat model—including Byzantine clients—describe the cryptographic and protocol-level countermeasures, and argue the safety and liveness guarantees provided by the system.

Keywords: Blockchain · HotStuff · EVM · Smart Contracts · Byzantine Fault Tolerance.

1 Introduction

The DepChain system implements a permissioned, append-only blockchain with high dependability guarantees in a closed, static membership environment. A predefined Public Key Infrastructure (PKI) supplies self-generated ECDSA key pairs and threshold signature key shares that are pre-distributed to every participant before the system starts.

The architecture separates *blockchain members* (servers), which maintain replicated state and run the consensus protocol, from a *client library* that translates application-level actions into signed requests broadcast to all members. The key challenge is bridging the client's simple interface with the BFT consensus primitives while ensuring that no single Byzantine entity—be it a compromised replica, a malicious leader, a network adversary, or a Byzantine client—can break safety or permanently prevent progress. With the integration of a Turing-complete EVM execution layer, the system now processes ordered blocks

containing multiple transactions, handles smart contract deployments, and defends against application-layer attacks such as resource exhaustion and Approval Frontrunning.

Section 2 describes the layered architecture. Section 3 details the improvements made to the first-stage design. Section 4 analyses the threat model and countermeasures. Section 5 argues the dependability guarantees.

2 System Architecture and Design

The system is organized into three strictly layered modules: the network layer, the consensus layer, and the execution and service layer.

2.1 Network Layer: Authenticated Perfect Links

Communication relies on raw UDP (Fair Loss Links). On top of UDP, an *Authenticated Perfect Links* (APL) abstraction provides:

- **Reliable delivery.** Every sent message is retransmitted until the destination sends back a sequence-numbered ACK.
- **No duplication and FIFO ordering.** A per-sender expected-sequence counter and an ordered delivery buffer ensure that messages are delivered exactly once and in sender order.
- **Authenticity.** Data and ACK packets carry an HMAC-SHA256 tag computed with a session key derived from an ECDH key agreement authenticated by ECDSA signatures from the static PKI, ensuring forward secrecy and mutual authentication.

2.2 Consensus Layer: Basic HotStuff

The consensus engine implements Algorithm 2 of the Basic HotStuff protocol [1]. Views are numbered sequentially with round-robin leader election. Each view has four phases (Prepare, Pre-Commit, Commit, Decide) with $O(n)$ message complexity. The protocol leverages *Quorum Certificates* (QC) proving that at least $2f+1$ replicas voted for a block using Shoup's threshold signature scheme [2]. Block identity is content-based via SHA-256, and the threshold signature transitively covers all client ECDSA signatures embedded in the block's transactions.

Transaction Batching. To optimize throughput, the leader accumulates transactions during a batching window (2 s). Once the window closes, a *greedy frontier algorithm* selects transactions for the block: pending transactions are grouped into per-client priority queues sorted by request ID (nonce), and the algorithm iteratively picks the highest-fee transaction across all queue heads, respecting both per-client nonce ordering and a block gas limit of 250 000 units—chosen to accommodate approximately 11 standard transfers per block, ensuring fast deterministic execution and bounded block propagation delays. Any transaction whose `gasLimit` alone exceeds the block gas limit is discarded. The estimated fee used for ordering is $estimatedGas \times gasPrice$, where $estimatedGas$ accounts for intrinsic costs (21 000 for transfers, 53 000 for deployments) plus calldata costs.

Blockchain Recovery. If a node receives a Decide for a block whose parent chain has gaps in its local block store, it initiates a recovery protocol. The node

broadcasts a `FETCH-BLOCKS-REQ` containing the missing parent hash to all peers. Responders walk their own block store backward (up to 20 blocks) and return the chain. The recovering node collects $n-f$ matching responses, verifies that all responses agree on the same block hashes, validates every ECDSA client signature in the recovered blocks, stores the recovered blocks from oldest to newest, and then resumes execution. Recovery state survives view changes, and a 10-second timeout triggers re-attempts.

2.3 Execution Layer and EVM Integration

State management and transition logic is handled by the `ServiceState` and `UpcallHandler` components. DepChain utilizes the Hyperledger Besu EVM [5] (Cancun specification) to execute smart contracts deterministically.

Account Model. Following the Ethereum model, two types of accounts exist: Externally Owned Accounts (EOAs), identified by public/private key pairs, and Contract Accounts, created deterministically via `contractAddress(deployer, nonce)`. Each account has a balance (amount of DepCoin) and a nonce. Contract Accounts additionally store EVM bytecode and key-value storage.

Genesis Block. The system initializes from a `genesis.json` file that prefunds all EOAs (nodes and clients) with 1 000 000 DepCoin each and deploys the ISTCoin smart contract via a contract-creation transaction executed in a real Besu EVM frame with 30 000 000 gas.

Transaction Execution and Gas. Clients submit transactions specifying a target address, value, EVM bytecode payload, gas limit, and gas price. The `UpcallHandler` employs a *two-level updater architecture* for atomicity:

- A *top-level updater* manages fee deduction and nonce increments—these persist even if EVM execution reverts.
- A *child updater* isolates EVM state changes. On success it is committed upward; on failure it is discarded, achieving automatic rollback.

Transactions with a `gasLimit` or `gasPrice` of zero are immediately rejected to prevent free network spam. Before execution, the upfront cost ($gasLimit \times gasPrice + value$) is verified against the sender's balance. The actual fee charged is $\min(gasLimit, gasUsed) \times gasPrice$; if execution reverts, the full $gasLimit \times gasPrice$ is charged. Gas fees are credited to the block's miner (the view leader). The system supports three execution paths: native balance queries (21 000 intrinsic gas), simple DepCoin transfers (21 000 gas), and full EVM execution for smart contract calls and deployments. Read-only operations such as balance queries are routed through the consensus layer rather than bypassing it; this ensures that clients always receive strongly consistent and finalized state reads, trading minor latency for strict linearizability. Consequently, because these queries are processed as standard consensus transactions that consume intrinsic gas, the returned value reflects the account balance immediately before the fee deduction of the query itself.

Block Persistence. Committed blocks are stored as individual JSON files per node, maintaining a uniform schema that strictly mirrors the structure of the genesis block. Each file contains the block hash (SHA-256), the previous block hash, a full state snapshot of all tracked accounts, and the serialized transactions

array. This array ensures total traceability of all network interactions, including read-only operations. For instance, when a client performs a native balance query, the operation is explicitly recorded in the block: the `from` field captures the address of the querying account, while the `to` field registers the address of the account being queried. This granular logging of state-viewing actions justifies the intrinsic gas fee deduction and provides a comprehensive, non-repudiable audit trail of all client activities.

2.4 Smart Contracts: ISTCoin

DepChain hosts the *ISTCoin* token, an ERC-20 fungible token [3] based on the OpenZeppelin reference implementation [4], written in Solidity with an initial supply of 100 000 000 units and 2 decimal places. The total supply is distributed equally among the client accounts at genesis. Standard ERC-20 implementations are vulnerable to *Approval Frontrunning*: when a client reduces a spender's allowance (e.g., from 100 to 50 tokens), the spender can front-run the approval by spending the original 100 tokens before the allowance is lowered, then spending the new 50, for a total of 150 tokens instead of 50.

To mitigate this, the ISTCoin contract overrides the `approve` function to require that either the new value or the current allowance is zero before any modification:

```
require(value == 0 || allowance(msg.sender, spender) == 0)
```

This forces a two-step process (reset to zero, then set new value) that eliminates the frontrunning window. Additionally, ISTCoin provides `increaseAllowance` and `decreaseAllowance` functions for atomic, safe allowance adjustments without exposure to the race condition.

3 Stage 1 Improvements

After the first stage, four improvements were made to strengthen the system before proceeding with the second stage requirements.

3.1 Block-Level Signature Verification

The original design maintained a `knownRequests` map inside the consensus engine: each node populated the map upon receiving a valid client request, and replicas only voted for proposals whose commands appeared in this registry. This approach was replaced with *block-level ECDSA signature verification*. Each `ClientRequest` carries an ECDSA signature over its canonical fields. When a replica receives a PREPARE message, the `BlockchainMember` layer verifies every client signature in the proposed block before the message reaches the consensus engine. Because the threshold signature is computed over the block's hash, it inherently secures all transactions inside it. Thus, a valid QC acts as a cryptographic proof that $2f+1$ replicas successfully and independently verified the ECDSA signature of every embedded client request. This eliminated the need for synchronized dual tracking between the event loop and networking threads.

3.2 Full Blockchain Recovery

The original system had no mechanism for a node that missed earlier views to recover the missing blocks. The recovery protocol described in Section 2 was

added: upon detecting a gap in the local block store, a node fetches the missing chain from its peers, validates the responses against a quorum of $n-f$ agreeing nodes, verifies all client signatures in recovered blocks, and then resumes normal execution. This ensures that temporarily disconnected or slow nodes can safely catch up without compromising the integrity of their local ledger.

3.3 FIFO-Ordered Message Delivery

The original APL implementation used a sliding-window deduplication approach that delivered messages immediately upon reception, regardless of order. This was replaced with a *sequence-ordered delivery buffer*: a per-sender expected-sequence counter tracks the next expected sequence number, and out-of-order packets are buffered. Contiguous packets starting from the expected sequence are drained and delivered in order. A gap timeout (1.5 s) prevents permanent blocking if a packet is lost, by skipping to the lowest pending sequence number and delivering from there. This provides FIFO per-sender delivery, which is essential for maintaining correct request ordering across the system.

3.4 Request ID Ordering and Deduplication

A multi-layered deduplication mechanism was introduced. At the application layer, a per-client monotonically increasing `nextExpectedClientRequestId` counter ensures that only the next expected request ID is accepted from each client—out-of-order or duplicate requests are dropped. At the execution layer, an `executedRequestIds` set prevents any request from being executed twice, even if a Byzantine leader attempts to replay it. Additionally, when batching transactions, the leader's greedy frontier algorithm enforces per-client nonce ordering by consuming each client's transactions via a priority queue sorted by request ID.

4 Threat Analysis and Protection Mechanisms

We consider an adversary that controls up to f of the $n = 3f+1$ blockchain members and any number of Byzantine clients. Clients cannot subvert the keys of honest clients.

4.1 Network-Level Attacks

Unreliable channel. UDP provides no delivery guarantees. The APL layer compensates with persistent retransmission, HMAC-SHA256-authenticated ACKs, and FIFO-ordered delivery via sequence-tracked buffers.

Message injection, spoofing, and MITM. HMAC-SHA256 with ECDH-derived session keys renders injected packets unforgeable. The ECDH handshake is authenticated with static ECDSA signatures, preventing man-in-the-middle attacks. HMAC verification uses constant-time comparison to prevent timing side-channels.

4.2 Byzantine Leader Attacks

QC forgery. Forming a valid Quorum Certificate requires $2f+1$ threshold signature shares; the adversary controls at most f.

Fabricated or modified transactions. Every replica independently verifies all client ECDSA signatures in a proposed block before voting. A leader cannot introduce transactions that no client signed.

Equivocation. The `safeNode` locking predicate ensures that a replica only votes for a block that extends its `lockedQC` or carries a justifying QC from a strictly higher view. Combined with the quorum intersection property ($|Q_1 \cap Q_2| \geq f+1$), two conflicting blocks can never both be committed.

Transaction replay. Before voting on a PREPARE, each node checks whether any transaction in the block has already been executed (via the `executedRequestIds` set). This prevents a Byzantine leader from re-proposing committed transactions.

4.3 Byzantine Client Attacks

Transaction forgery and non-repudiation. Every `ClientRequest` includes an ECDSA signature over its canonical fields (client ID, request ID, target address, value, gas parameters, payload, and timestamp). This signature is verified at three levels: upon network reception, before consensus voting (block-level verification), and again before EVM execution (defence-in-depth). This guarantees strict non-repudiation—a Byzantine client cannot spend funds from an honest client's account.

Replay attacks. DepChain mitigates replays at two independent levels: the application layer enforces strictly incrementing request IDs per client, and the execution layer maintains an `executedRequestIds` set that prevents double execution of any transaction.

Balance subversion. Before any state mutation, the `UpcallHandler` verifies that the sender's balance covers the upfront cost ($gasLimit \times gasPrice + value$). The two-level updater architecture ensures that failed transactions do not leave the state in an inconsistent state. Balances are strictly non-negative.

Resource exhaustion (DoS). A client could deploy a smart contract with an infinite loop to halt the blockchain. The gas mechanism mitigates this: every EVM operation consumes predefined gas units, and once the `gasLimit` is exhausted, execution is forcefully halted and state changes are reverted. The client is still charged for the gas consumed, making DoS attacks economically costly.

Approval Frontrunning. The ISTCoin contract's modified `approve` function (Section 2.4) prevents a Byzantine client from exploiting the allowance race condition to drain more tokens than authorized.

4.4 Client–Server Collusion

Fee evasion. A Byzantine client colluding with a Byzantine leader could attempt to have the leader propose the client's transaction with manipulated gas parameters (e.g., zero fees). This attack fails because each honest replica independently re-verifies the client's ECDSA signature over the original transaction fields—including the gas limit and gas price—before voting on the PREPARE. Any modification to these fields invalidates the signature, preventing the proposal from gathering $2f+1$ votes. Furthermore, every replica executes the EVM deterministically and independently computes the gas fee, so even if the leader attempts to credit a colluding client, the honest replicas will produce a divergent state and refuse to sign the QC.

Selective inclusion. A colluding leader might prioritize a Byzantine client's transactions or censor honest ones. However, leader rotation guarantees that honest leaders eventually process all pending transactions. The liveness properties of BFT consensus ensure that censorship is bounded to at most f consecutive views.

4.5 Recovery Attacks

Forged recovery blocks. When a node recovers missing blocks from peers, every block in the response is verified for valid client ECDSA signatures. Additionally, the recovering node requires $n-f$ consistent responses (same block hashes at each position), ensuring that a minority of Byzantine responders cannot inject forged blocks. Responses containing blocks with invalid signatures or inconsistent hashes are discarded.

5 Dependability Guarantees

5.1 Safety

Agreement. The three-phase HotStuff locking mechanism ensures that if a correct replica commits block b in view v, no correct replica can commit a conflicting block. Two quorums of $2f+1$ out of $3f+1$ always intersect in at least one honest node, making equivocation impossible.

Validity and integrity. Only client commands carrying verifiable ECDSA signatures alter the state. Replicas use deterministic EVM processing, ensuring identical transaction outputs across the network. Blocks are cryptographically chained using SHA-256 hashes, ensuring ledger immutability. Clients verify responses via the `CommitProof` threshold signature, which cryptographically proves consensus finality without relying on individual replica trustworthiness.

Non-repudiation. Every state modification is traceable to a transaction carrying a client ECDSA signature, verified at multiple layers. Clients cannot deny having issued a transaction once it is committed with their valid signature.

Non-negative balances. The upfront cost verification and two-level updater architecture guarantee that no account balance can become negative, regardless of Byzantine behaviour.

5.2 Liveness

Liveness (termination) is guaranteed after the Global Stabilization Time (GST), when network delays become bounded.

View-change mechanism. A configurable view timer fires if the leader fails to drive consensus to completion. Replicas time out and advance to the next view. Exponential backoff (doubling the timeout after each failed view, capped at 32 s) prevents livelock during transient instability, and the timeout resets upon successful completion.

Leader rotation. Round-robin leader election ($v \bmod n$) ensures every non-faulty replica eventually becomes the leader, bypassing Byzantine leaders within $f+1$ view changes.

Blockchain recovery. Temporarily disconnected nodes recover missing blocks via the quorum-validated fetch protocol, preventing them from falling permanently behind and maintaining the effective quorum size of $2f+1$.

6 Conclusion

DepChain successfully merges a resilient BFT consensus engine with a fully featured, deterministic execution environment. The first-stage foundation—threshold-verified quorum certificates, authenticated perfect links, and ECDSA-signed client requests—was strengthened with block-level signature verification, full blockchain recovery, FIFO-ordered delivery, and multi-layered request deduplication. Building upon this, the second stage introduced a complete distributed ledger with EVM-powered smart contract execution, native cryptocurrency transfers with gas metering, fee-based transaction batching, and a frontrunning-resistant ERC-20 token. The multi-layered defence-in-depth approach ensures uncompromised safety and liveness against both Byzantine servers and clients, fulfilling all requirements for a highly dependable distributed system.

References

1. Yin, M., Malkhi, D., Reiter, M. K., Gueta, G. G., Abraham, I.: HotStuff: BFT Consensus in the Lens of Blockchain. In: Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC '19), pp. 347–356. ACM (2019).
2. Shoup, V.: Practical Threshold Signatures. In: Advances in Cryptology — EUROCRYPT 2000, LNCS, vol. 1807, pp. 207–220. Springer (2000).
3. Vogelsteller, F., Buterin, V.: ERC-20 Token Standard. Ethereum Improvement Proposals, no. 20 (2015). `https://eips.ethereum.org/EIPS/eip-20`
4. OpenZeppelin: ERC20 Implementation. GitHub Repository. `https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/token/ERC20`
5. Hyperledger Besu Ethereum Client. Linux Foundation. `https://github.com/hyperledger/besu`